Test Suite Manager Requirements

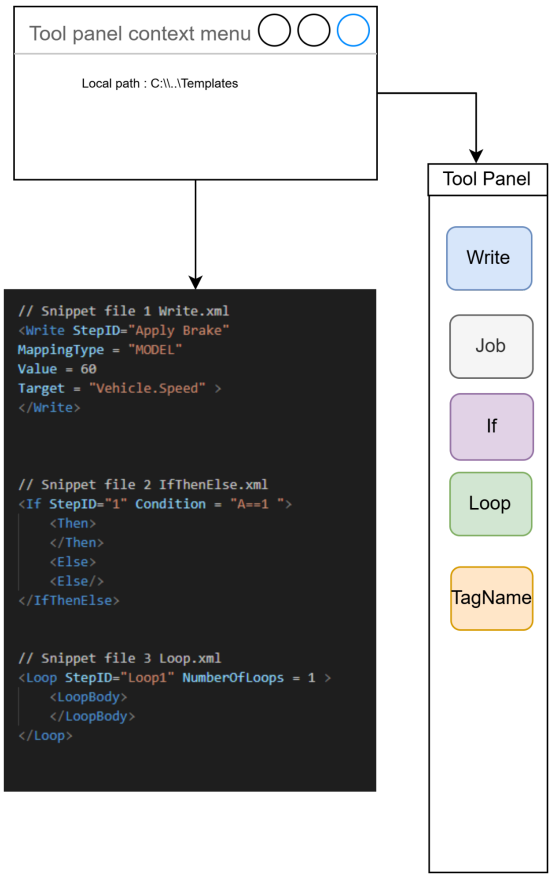
What is Test Suite Manager

The Test Suite Manager is an XML-oriented authoring tool designed to create, manage, and generate automotive test cases derived from manufacturer requirements.
It provides a structured yet flexible environment to visually construct test cases using reusable XML templates, ensuring consistency and efficiency across projects.

1. Test-case authoring

The tool allows flexible authoring of XML-based test cases supporting various test topologies. A **Tool Panel** provides a library of reusable XML templates that can be inserted into the **Test Case Edit Window** through drag-and-drop interaction.

1.A Tool Panel



The Tool Panel contains a configurable list of local XML templates, each representing a reusable ECU-TEST step (e.g., *Read, Write, Job, Calculation, Loop*)

- ➢ The tool panel must be scrollable to accommodate all template steps.
- ➢ Each template is displayed with an **icon and tag name** for quick identification.
- ➢ The **template directory path** must be configurable through a context menu (or equivalent setting).
- ➢ The configuration must be **persisted non-volatilely** between sessions.

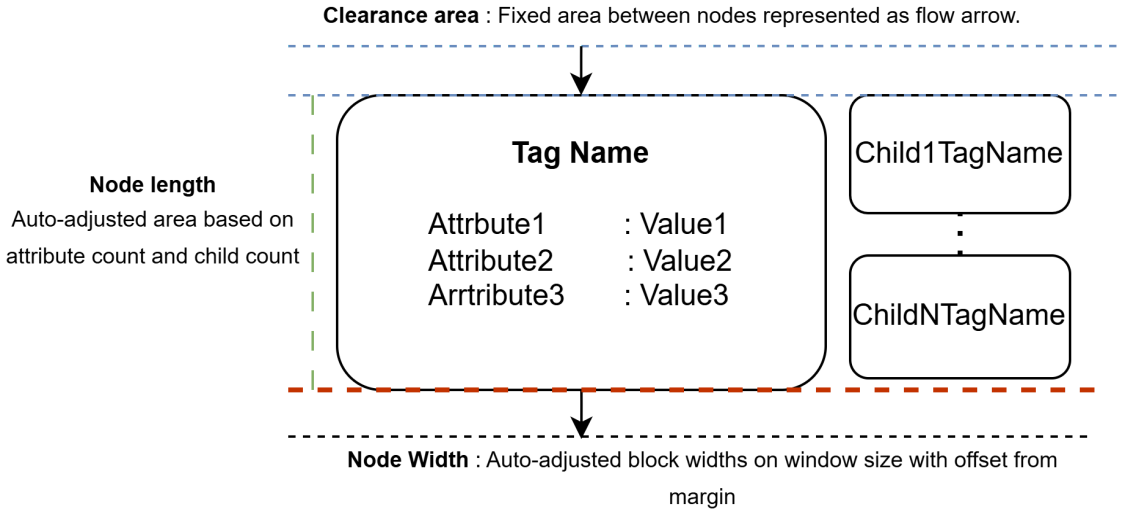
1B. Test Case Edit window

The Test Case Edit Window is the main user workspace for assembling and editing XML-based test cases.
Users can drag templates from the Tool Panel into this window to build hierarchical XML structures.

1B.1 Node representation and insertion behaviour

This window user interacts to drag the templates from the tool panel and edit the values.

- ➢ Each inserted XML template from the Tool Panel appears as a graphical node block within the edit window.



- ➢ Nodes are positioned **above or below** existing nodes depending on cursor position relative to the node's midpoint during drop.
- ➢ When inserted, the node block automatically adjusts its **width and height** based on:
  - The number of attributes

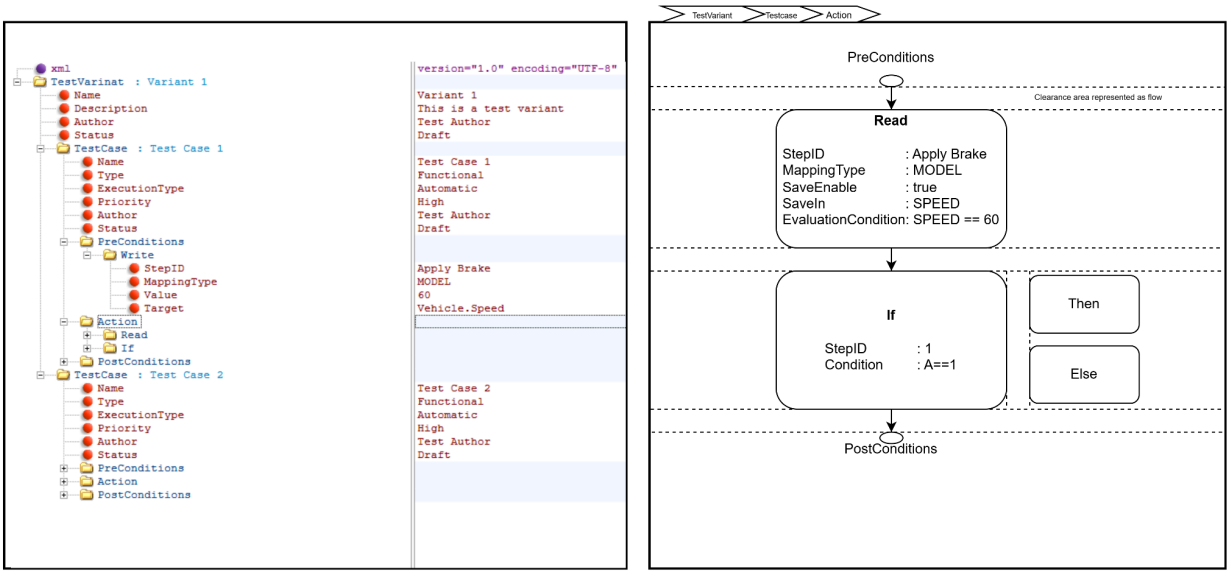
- The number of child elements

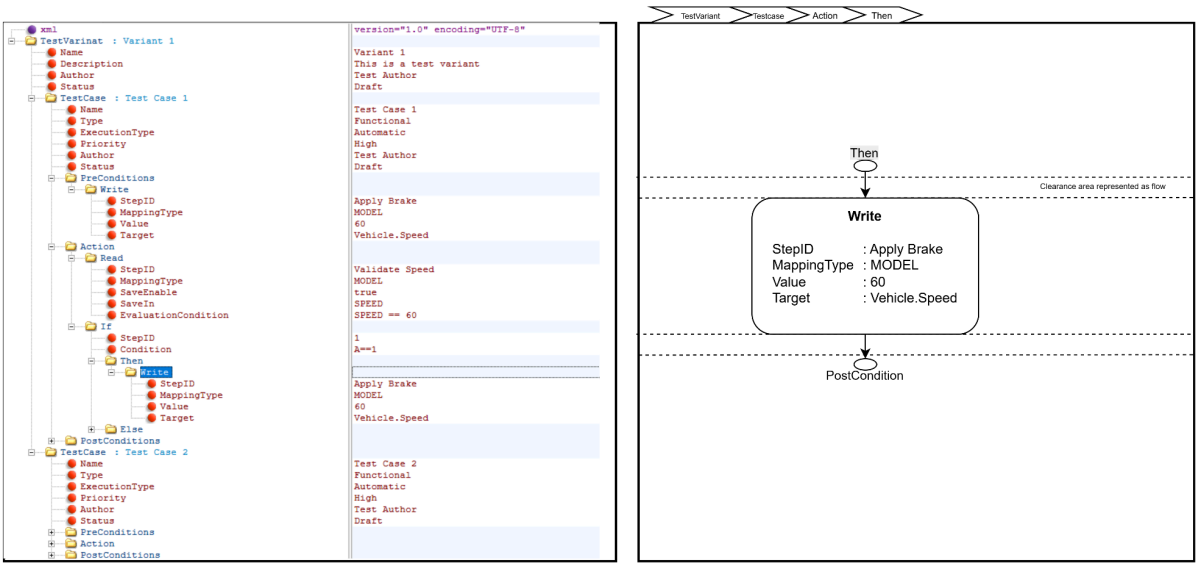
➢ Each node block should visually display:
  - The **tag name**
  - Key **attribute value pairs** (similar to masked subsystems in Simulink).

➢ Nodes may be **resized dynamically** if attributes or child nodes are added or removed.

1B.2 XML DOM/ Subsystem navigation

This feature provides a Simulink-style hierarchical navigation system, allowing users to explore XML structures visually as nested subsystems.





Requirements:

A. **Hierarchy Representation**

a. Parent XML elements (e.g., `<TestCase>`, `<PreCondition>`, `<Action>`, `<Specific_TestSteps>`) are displayed as **parent blocks** or “subsystems.”
b. Each parent block visually contains its **child nodes**, represented as smaller blocks inside.

B. **Navigation Behavior**

a. **Double-clicking** a parent block opens its child context (similar to entering a subsystem in Simulink).
b. The view transitions smoothly to display only the **child nodes** within that parent.
c. A **breadcrumb trail** or **Back button** allows users to navigate to the previous parent level.
d. The navigation path should be clearly visible (e.g., `/TestCase/Action/WriteStep`).

C. **Synchronization with XML DOM**

a. The block hierarchy directly maps to the underlying **XML DOM structure**.
b. The navigation stack (current node context) should always point to the **corresponding XML element** in the DOM.

D. **View Layout and Interaction**

a. The edit window must support **zoom** operation, this should follow **Ctrl + Mouse wheel** operation, shrinking the block size, areas and font.
b. A scroll bar on the side allows the user to quickly maneuver large structures.
c. Blocks within a subsystem should be **auto-arranged** by the mentioned format to avoid overlap.
d. When a new child node is inserted, it appears inside the current subsystem context and becomes part of that parent's DOM branch.

E. **Consistency and Undo/Redo**

a. All navigation and editing actions (enter, back, insert, delete) should be logged in the global undo/redo stack.
b. On returning from a subsystem view, the previous viewport and selection state should be restored.

1B.3 Attribute value edit

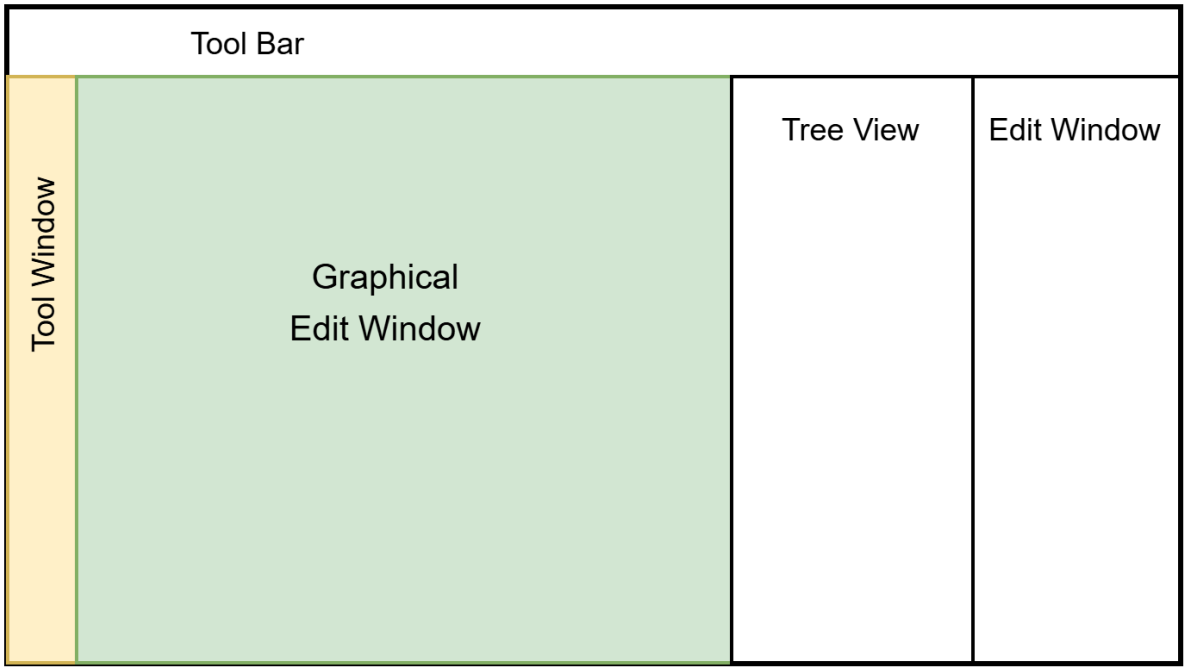
The user must be able to edit the attribute values of a graphical node (block) directly within the visual block itself or through a dedicated pop-up editor window (developer's implementation choice)

- User entered value should be validated against the predefined conditions described in the TestStepRule.json file.
  - Validation must support:Regular expression (`Regex`) matching.
  - Enumerated option lists (`Options` arrays).
  - Dependency rules (`DependsOn`.).
- Invalid inputs must trigger clear visual feedback (e.g., red outline, tooltip with `ErrorMessage` from JSON).

- Each editable field should support **autocomplete/autofill suggestions(List)** based on the corresponding JSON definition.
- The path to the `TestStepRule.json` file must be **user-configurable** from the test tool's main **Configuration Panel**.

Any improvement and suggestions from the developer on this feature is appreciated.

Final window outlook



2. Test Case Management

In short the tool should maintain the above functional features back to back.

- ➢ Any edits (add, delete, rename) in the visual view must immediately reflect in the XML tree and vice versa.
- ➢ Any XML pasted to the Xml Notepad branch view must change the DOM Structure (available feature maintained) and **reflect in the edit window** gui and vice versa.

3. Test Case Generation

Developer may suggest integration methods for python application to the existing tool.